



湖南大学
HUNAN UNIVERSITY

第一章 算法设计与分析基础

—— 湖南大学计算机学院 ——

联系方式



任课教师：彭 鹏

办公室：软件大楼536

邮箱：hnu16pp@hnu.edu.cn

教材以及课程资源



教材:

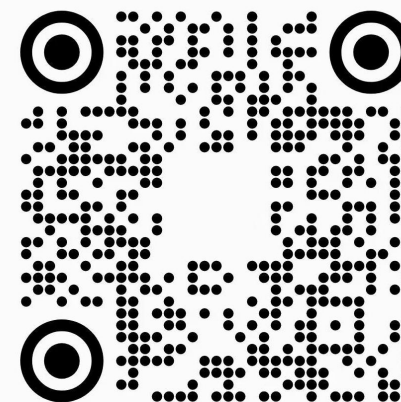
算法导论, [美] Thomas H.Cormen, [美] Charles E.Leiserson, [美] Ronald L.Rivest, [美] Clifford Stein 著, 殷建平, 徐云, 王刚 等 译

课程网站:

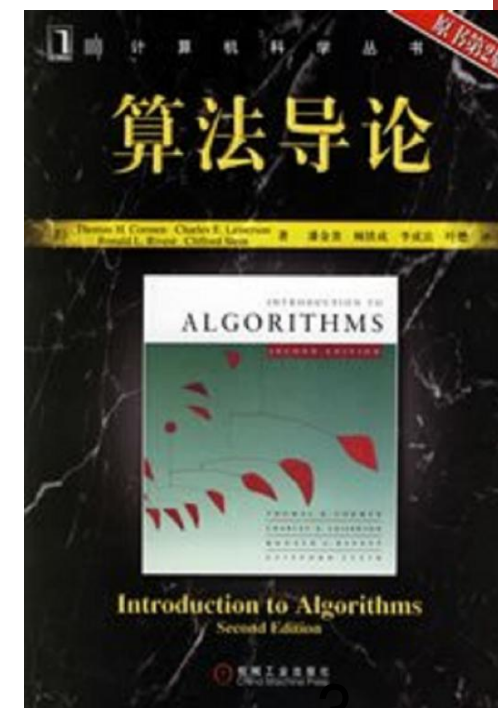
<https://bnu05pp.github.io/AlgorithmDesign/index.html>



算法设计与分析-2026
群号: 651626178



扫一扫二维码, 加入群聊

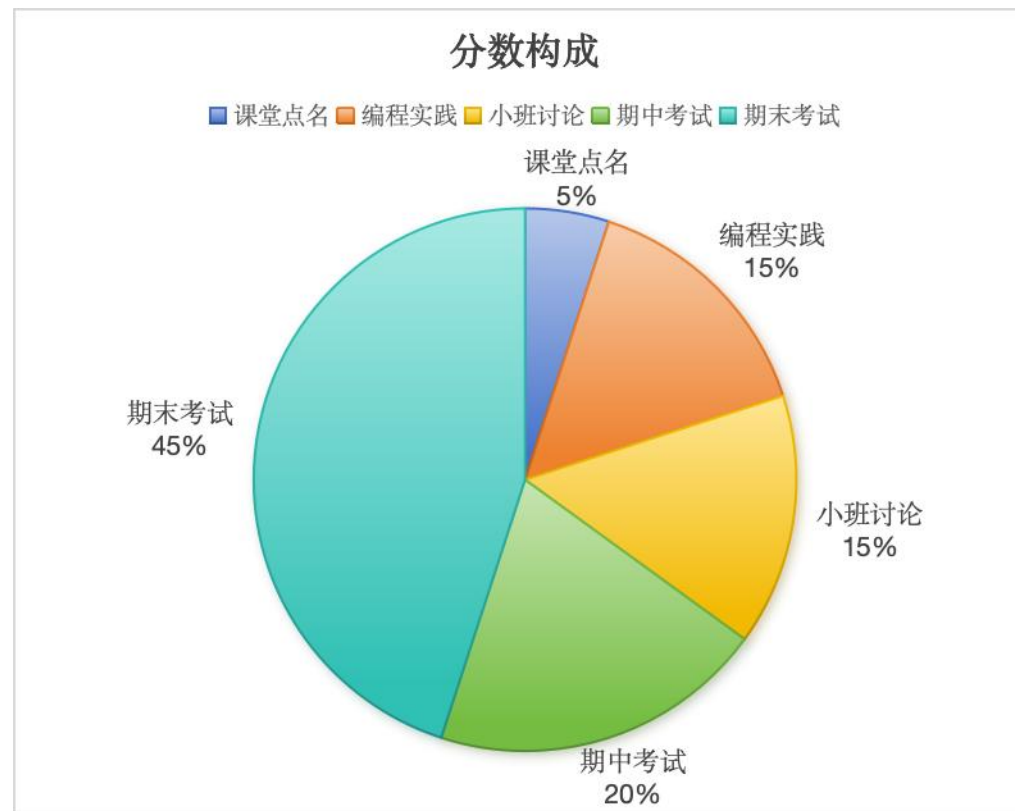


考核方式



考核形式:

- (1) 课堂点名 (占5%)
- (2) 编程实践 (占15%)
- (3) 小班讨论 (占15%)
- (4) 期中考试 (占20%)
- (5) 期末考试 (占45%)



编程实践考核说明



- 每周周五8:00到24:00之间，将自己Leetcode截屏发给我学生邮箱3318965636@qq.com，邮件标题格式为"Leetcode截屏-[姓名]-[学号]-[日期]"
- 每周根据题目计分，题目范围参照课程进度。一个难度简单的题，计2分；一个难度中等的题，计3分；一个难度困难的题，计4分。单次分数上限10分。



小班讨论考核说明



- 小班讨论总分为100分，其中算法分析演讲60分，提交算法分析报告40分（算法分析报告发给我学生邮箱3318965636@qq.com）。
- 每班分6组，5~6人一组。需要完成的任务要求如下：
- 每组需要运行代码，制作PPT讲解代码，分析每个程序的时间复杂度，程序的瓶颈及改进方式，每一组至少要安排2个不同的两个人进行讲解。
- 详细参见：<https://bnu05pp.github.io/AlgorithmDesign/pingshichengji.html>

目录

第一节

背景介绍

第二节

算法基本概念

第三节

算法时间复杂度分析

第四节

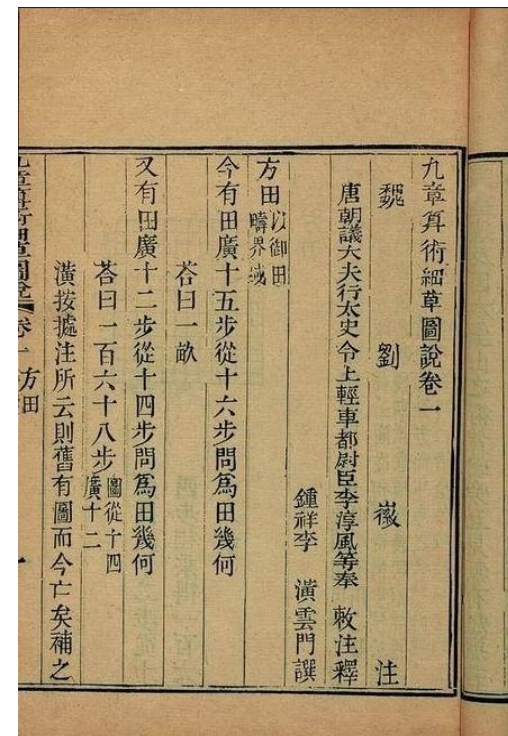
一个算法例子： 文本距离

算法历史



□ “算法”一词的来历

- 中文的“算法”：《九章算术》
- 英文的“算法” (algorithm)：花拉子米 (al-Khwārizmī)
- 图灵机





□ 二十世纪十大著名算法

- 蒙特卡洛方法：1946年
- 单纯形法：1947 年
- Krylov子空间迭代法：1950年
- 矩阵计算的分解方法：1951 年
- 优化的Fortran编译器：1957年
- 计算矩阵特征值的QR算法：1959-61 年
- 快速排序算法：1962年
- 快速傅立叶变换：1965年
- 整数关系探测算法：1977年
- 快速多极算法：1987年

算法——科技巨头的灵魂



- 百度、谷歌、字节它们的核心竞争力是什么？
- 百度/谷歌 的强大，不在于它们存储了多少网页对象，
 - 而在于它们如何用算法在茫茫数据海洋中瞬间找到你要的那一滴水。
- 抖音/字节 的成功，不在于它们播放了多少视频实例，
 - 而在于它们如何用算法比你更懂你，精准预测你的下一次点击。
- 算法学习的目标
 - 不要只做会写 Class 和 Object 的码农。
 - 要做能设计高效解决海量数据难题的算法工程师；
 - 算法能力，是你未来职业生涯最宽的护城河。

目录

第一节

背景介绍

第二节

算法基本概念

第三节

算法时间复杂度分析

第四节

一个算法例子：
文本距离



□ 广义算法

➤ 算法是解决问题的方法，如一道菜谱、一个安装电脑的操作指南等。

□ 狭义算法：计算机算法

➤ 计算机算法是对特定问题求解步骤的一种描述，是指令的有限序列。



□ 算法的五个重要特性

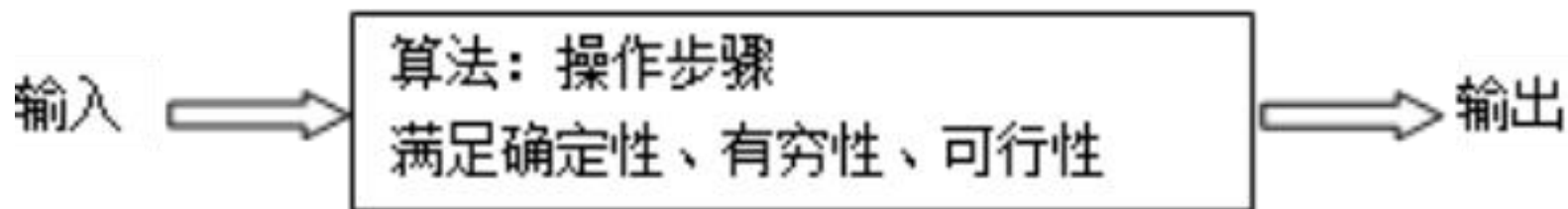


图 1.1 算法的概念



□ “好” 算法的五个其他特性

- 正确性
- 健壮性
- 可理解性
- 抽象分级
- 高效性



- 清晰的形式化描述，是评估算法优劣与设计高效方案的前提。
- 我们将通过三种视角来审视算法描述：
 - 自然语言
 - 程序设计语言
 - 伪代码



□ 算法的描述方法

➤ 自然语言

➤ 程序设计语言

➤ 伪代码

问题：自然语言有歧义

武松打死老虎

武松/打死老虎

武松/打/死老虎

$\text{sum} = 1 + 2 + 3 + 4 + 5 + \dots + (n-1) + n$ 为例

- ① 确定一个n的值;
- ② 假设等号右边的算式项中的初始值i为1;
- ③ 假设sum的初始值为0;
- ④ 如果 $i \leq n$ 时, 执行⑤, 否则转出执行⑧;
- ⑤ 计算sum加上i的值后, 重新赋值给sum;
- ⑥ 计算i加1, 然后将值重新赋值给i;
- ⑦ 转去执行④;
- ⑧ 输出sum 的值, 算法结束。



□ 算法的描述方法

- 自然语言
- 程序设计语言 (代码)
- 伪代码



□ 算法的描述方法

- 自然语言
- 程序设计语言
- 伪代码

算法开始

输入 n 的值;

可以加注解

$i \leftarrow 1$;

/*为 i 赋初值
*/

$s \leftarrow 0$;

/*为 s 赋初值*/

While($i \leq n$)

/*循环语句*/

{

/*循环开始*/

$s \leftarrow s + i$;

/*把 i 累加到 s */

$i \leftarrow i + 1$;

/*记数*/

}

/*循环结束*/

输出 s 的值;

算法结束

算法基本概念



□ 算法设计的一般过程

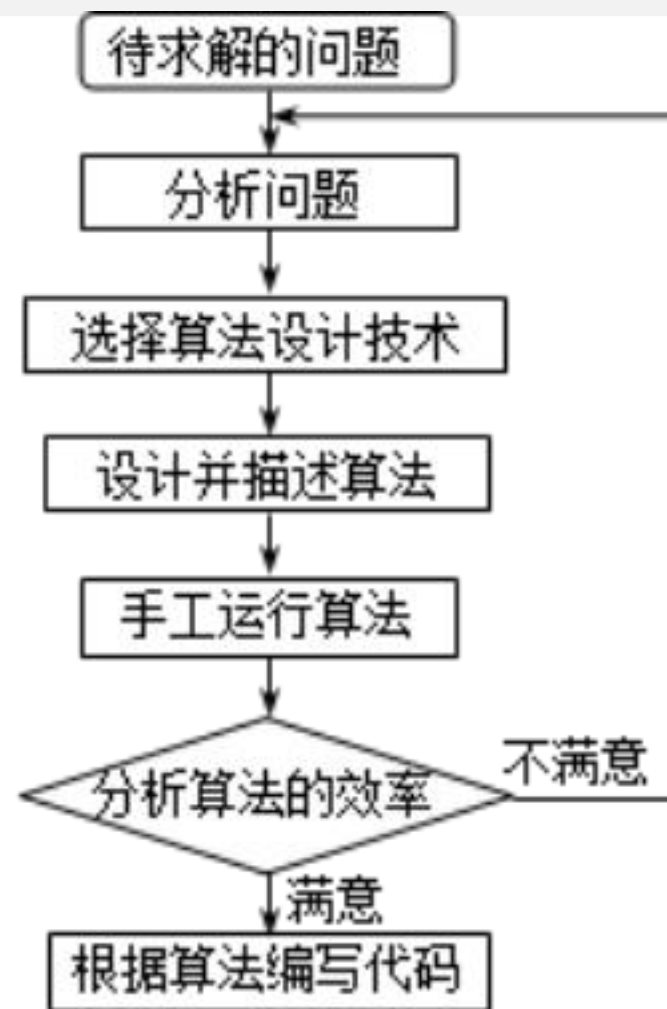


图 1.2 算法设计的一般过程



□ 算法与数据结构

- 数据结构是底层，算法高层。数据结构为算法提供服务。算法围绕数据结构操作。

□ 算法与程序

- 程序是某个算法或过程的在计算机上的一个具体的实现。
- 程序是依赖于程序设计语言的，甚至依赖于计算机结构的。
- 算法是脱离具体的计算机结构和程序设计语言的。

重要问题类型



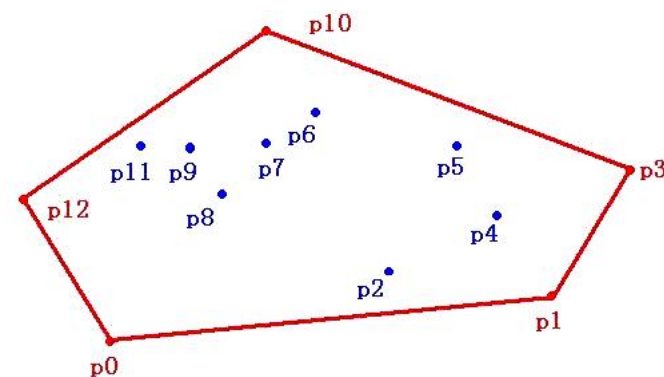
□ 查找问题

□ 排序问题

□ 图问题

□ 组合问题

□ 几何问题



目录

第一节

背景介绍

第二节

算法基本概念

第三节

算法时间复杂度分析

第四节

一个算法例子：
文本距离

算法复杂性分析



□ 影响算法性能的因素？

影响算法性能的因素——数据规模



10种排序算法的时间效率比较

算法\输入数据	N=50 K=50	N=200 K=100	N=500 K=500	N=2000 K=2000	N=5000 K=8000	N=10000 K=20000	N=20000 K=20000	N=20000 K=200000
冒泡排序	0ms	15ms	89ms	1493ms	9363ms	36951ms	147817ms	143457ms
插入排序	1ms	13ms	82ms	1402ms	8698ms	34731ms	134817ms	134836ms
希尔排序	0ms	1ms	6ms	30ms	110ms	257ms	599ms	606ms
选择排序	0ms	5ms	31ms	461ms	2888ms	11736ms	45308ms	44838ms
堆排序	0ms	3ms	9ms	40ms	124ms	247ms	525ms	527ms
归并排序	2ms	6ms	18ms	75ms	199ms	392ms	778ms	793ms
快速排序	0ms	1ms	2ms	14ms	36ms	84ms	196ms	163ms
计数排序	0ms	1ms	1ms	5ms	15ms	32ms	51ms	62ms
基数排序	0ms	1ms	4ms	19ms	47ms	114ms	237ms	226ms
桶排序	0ms	2ms	6ms	25ms	68ms	126ms	254ms	251ms

影响算法性能的因素——操作数量



□ 算法的好与坏

➤ 高斯的故事

$$1+2+3+4+5+6+\cdots+100 = ?$$

$$\begin{aligned} & (1+100) + (2+99) + \cdots + (50+51) \\ & = 50 \times 101 \quad (\text{高斯算法}) \end{aligned}$$



高斯（数学王子）

高斯：“给我最大快乐的，不是已懂得知识，而是不断的学习；不是已有的东西，而是不断的获取；不是已达到的高度，而是继续不断的攀登。”

算法复杂性分析



- 是关于计算机程序性能和资源利用的研究
 - 时间复杂度分析
 - 空间复杂度分析

算法评价的核心：复杂性分析



- 核心观点：要科学判断算法优劣，必须摒弃主观感觉，将算法性能进行形式化量化。
- 目标：建立一套通用的数学标准，使不同算法可在同一维度下比较。
- 构建数学模型，精准描述算法资源消耗（时间/空间）随问题规模变化的增长规律。

问题规模与渐近时间复杂度



□ 定义阐述

- 问题规模 (n): 度量输入实例大小的参数 (如数组长度、矩阵维数)。
- 频度统计 ($T(n)$): 算法中基本操作的执行次数, 它是问题规模 n 的函数, 记作 $T(n)$ 。
- 分析主旨: 通过推导 $T(n)$ 的表达式, 揭示算法运行时间随输入规模变化的增长趋势。

□ 实例解析

- 案例 A: $T(n) = \log_2 n$
- 解读: 当输入规模为 n 时, 算法执行的基本操作次数呈对数增长。
- 案例 B: $T(n) = n^2$
- 解读: 当输入规模为 n 时, 算法执行的基本操作次数呈平方级增长。

算法复杂性分析--时间复杂度分析



□ 渐进分析

表 2-8-1

次数	算法 A ($2n + 3$)	算法 A' ($2n$)	算法 B ($3n + 1$)	算法 B' ($3n$)
$n = 1$	5	2	4	3
$n = 2$	7	4	7	6
$n = 3$	9	6	10	9
$n = 10$	23	20	31	30
$n = 100$	203	200	301	300

算法复杂性分析--时间复杂度分析



□ 渐进分析

表 2-8-2

次数	算法 C ($4n+8$)	算法 C' (n)	算法 D ($2n^2+1$)	算法 D' (n^2)
$n = 1$	12	1	3	1
$n = 2$	16	2	9	4
$n = 3$	20	3	19	9
$n = 10$	48	10	201	100
$n = 100$	408	100	20 001	10 000
$n = 1000$	4 008	1 000	2 000 001	1 000 000



渐进符号——运行时间的上界

若存在两个正的常数 c 和 n_0 ,对于任意 $n \geq n_0$,都有 $T(n) \leq cf(n)$,则称 $T(n) = O(f(n))$ 。

大 O 符号描述增长率的上限,表示 $T(n)$ 的增长最多像 $f(n)$ 增长的那样快,换言之,当输入规模为 n 时,算法消耗时间的最大值,这个上限的阶越低,结果越有价值。

该算法的运行时间至多是 $O(f(n))$ 。

若存在两个正的常数 c 和 n_0 ,对于任意 $n \geq n_0$,都有 $T(n) < cf(n)$,则称 $T(n) = o(f(n))$ 。



渐进符号——运行时间的上界

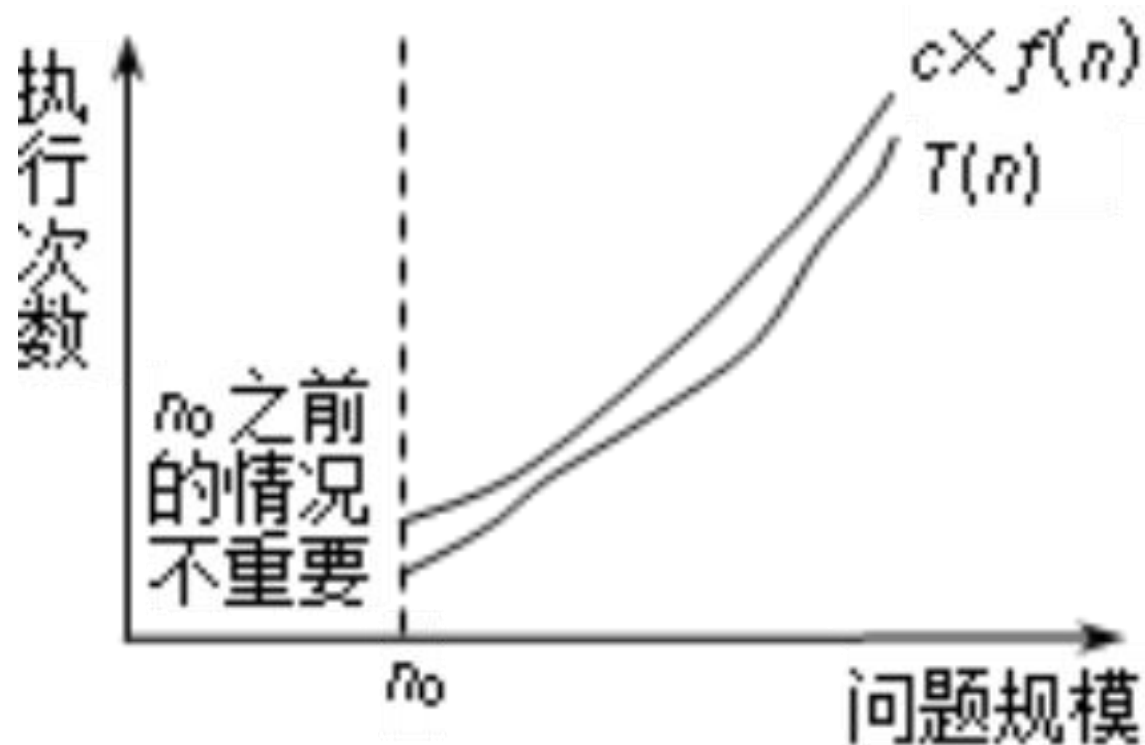


图 2.1 大 O 符号的含义



渐进符号——运行时间的下界

若存在两个正的常数 c 和 n_0 ,对于任意 $n \geq n_0$,都有 $T(n) \geq cg(n)$,则称 $T(n) = \Omega(g(n))$ 。

大 Ω 符号用来描述增长率的下限,也就是说,当输入规模为 n 时,算法消耗时间的最小值。与大 O 符号对称,这个下限的阶越高,结果就越有价值。

该算法的运行时间至少是 $\Omega(g(n))$ 。

若存在两个正的常数 c 和 n_0 ,对于任意 $n \geq n_0$,都有 $T(n) > cf(n)$,则称 $T(n) = \omega(f(n))$ 。



渐进符号——运行时间的下界

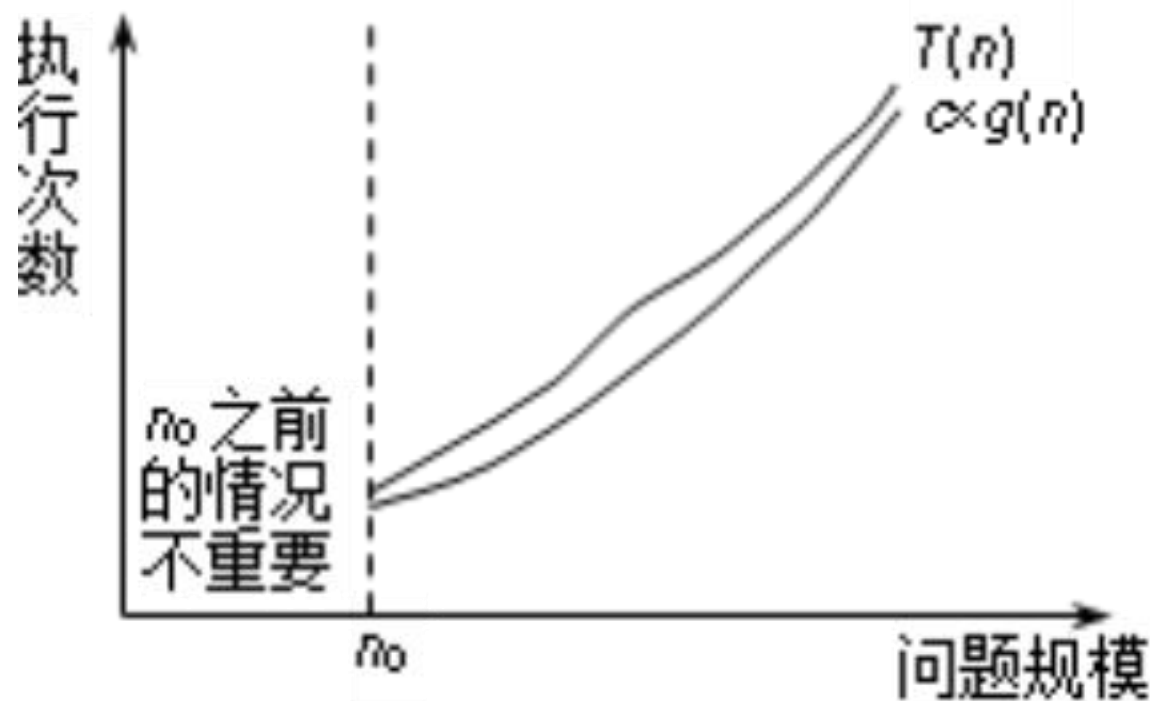


图 2.2 大 Ω 符号的含义



渐进符号——运行时间的下界

Θ 符号(运行时间的准确界)

定义

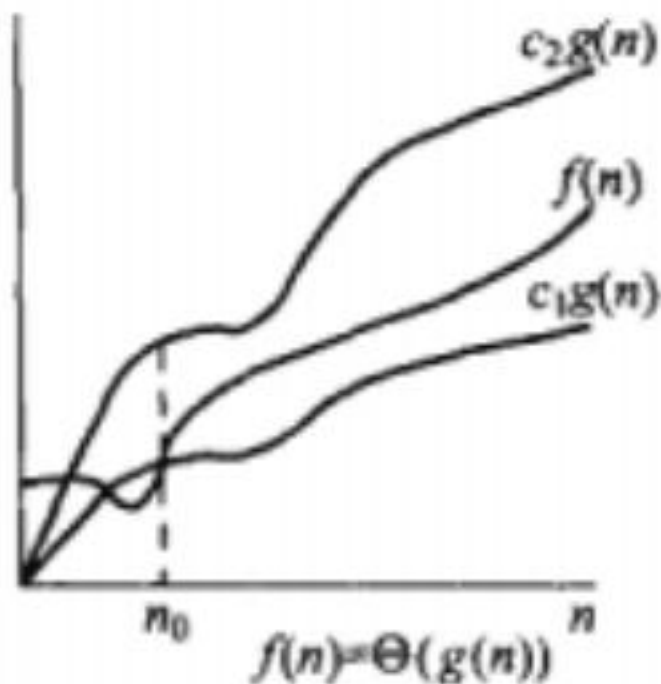
若存在三个正的常数 c_1 、 c_2 和 n_0 , 对于任意 $n \geq n_0$, 都有 $c_1g(n) \geq T(n) \geq c_2 \times g(n)$, 则称 $T(n) = \Theta(g(n))$ 。

Θ 符号意味着 $T(n)$ 与 $g(n)$ 同阶, 用来表示算法的精确阶。

算法复杂性分析--时间复杂度分析



渐进符号——运行时间的准确界



常见复杂度层级与渐近性质



□ 忽略常数与低阶项：当 $n \rightarrow \infty$ 时，主导项决定增长趋势。

➤ 例： $3n^2 + 100n + 5 = \Theta(n^2)$

➤ 例： $100 = \Theta(1)$; $1000 = \Theta(1)$; $10000 = \Theta(1)$

常见复杂度层级与渐近性质



□复杂度增长阶梯 (由优到劣)

➤ 对数阶的无关底数性: $\log_a n = \Theta(\log_b n)$

解读: 对数函数的底数变化仅影响常数倍, 量级不变。通常统一记为 $\log n$ 。

➤ 对数 vs 多项式: $\log n = O(n^k)$ (对于任意 $k > 0$)

解读: 对数增长远慢于任何正次幂的多项式。哪怕 $n^{0.001}$ 也比 $\log n$ 增长快。

➤ 多项式内部的层级: $n^k = O(n^m)$ (当 $k \leq m$)

解读: 指数越高, 增长越快。 $n < n^2 < n^3 \dots$

➤ 多项式 vs 指数: $n^k = O(2^n)$ (对于任意常数 k)

解读: 任何多项式增长都远慢于指数增长。

常见复杂度层级与渐近性质



□复杂度增长阶梯 (由优到劣)

➤ 指数内部的层级: $k^n = O(m^n)$ (当 $k \leq m$)

解读: 底数越高, 增长越快。 $2^n < 3^n < 4^n \dots$

➤ 指数 vs 阶乘: $2^n = O(n!)$

解读: 阶乘增长远超指数。



渐进符号——渐进比较

传递性

$$f(n) = \Theta(g(n)) \text{ and } g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n))$$

$$f(n) = O(g(n)) \text{ and } g(n) = O(h(n)) \Rightarrow f(n) = O(h(n))$$

$$f(n) = \Omega(g(n)) \text{ and } g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n))$$



渐进符号——渐进比较

自反性

$$f(n) = \Theta(f(n))$$

$$f(n) = O(f(n))$$

$$f(n) = \Omega(f(n))$$



渐进符号——渐进比较

对称性

$$f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$$

$$f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$$

$$f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$$

算法复杂性分析--时间复杂度分析



- 1.最好情况时间复杂度 (best case time complexity) 。
- 2.最坏情况时间复杂度 (worst case time complexity) 。
- 3.平均情况时间复杂度 (average case time complexity) 。
- 4.均摊时间复杂度 (amortized time complexity) 。

算法复杂性分析--时间复杂度分析



1. 最好情况时间复杂度 (best case time complexity) 。
2. 最坏情况时间复杂度 (worst case time complexity) 。
3. 平均情况时间复杂度 (average case time complexity) 。
4. 均摊时间复杂度 (amortized time complexity) 。

算法复杂性分析--时间复杂度分析



□ 非递归算法分析的一般步骤

1. 决定用哪个（或哪些）参数作为算法问题规模的度量

可以从问题的描述中得到。

2. 找出算法中的基本语句

通常是最内层循环的循环体。

3. 检查基本语句的执行次数是否只依赖于问题规模

如果基本语句的执行次数还依赖于其他一些特性，则需要分别研究最好情况、最坏情况和平均情况的效率。

算法复杂性分析--时间复杂度分析



□ 非递归算法分析的一般步骤

4. 建立基本语句执行次数的求和表达式

计算基本语句执行的次数，建立一个代表算法运行时间的求和表达式。

5. 用渐进符号表示这个求和表达式

计算基本语句执行次数的数量级，用大O符号来描述算法增长率的上限。



常用的级数公式及复杂度

- ❖ 算数级数： $T(n) = 1 + 2 + \dots + n = n(n+1)/2 = O(n^2)$
- ❖ 平方级数： $T(n) = 1^2 + 2^2 + \dots + n^2 = n(n+1)(2n+1)/6 = O(n^3)$
- ❖ 幂级数： $T_a(n) = a^0 + a^1 + a^2 + \dots + a^n, a > 0$
最常见： $1+2+4+\dots+2^n = 2^{n+1}-1 = O(2^{n+1}) = O(2^n)$ //总和与末项同阶
- ❖ 收敛： $1/1/2 + 1/2/3 + 1/3/4 + \dots + 1/(n-1)/n = 1 - 1/n = O(1)$
 $1 + 1/2^2 + \dots + 1/n^2 < 1 + 1/2^2 + \dots = \pi^2/6 = O(1)$
- ❖ 有必要讨论这类级数吗？难道，基本操作次数、存储单元数可能是分数？
- ❖ 调和级数： $h(n) = 1 + 1/2 + 1/3 + \dots + 1/n = \Theta(\log n)$
// $h(n) < 1 + \int_{1 \sim n} 1/x = 1 + \ln n = O(\log n)$
// $h(n) > \int_{1 \sim n+1} 1/x = \ln(n+1) = \Omega(\log n)$
- ❖ 对数级数： $\log 1 + \log 2 + \log 3 + \dots + \log n = \log(n!) = \Theta(n \log n)$

算法复杂性分析--时间复杂度分析



□ 小结:

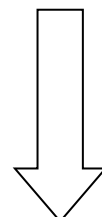
基本技术:

- ◆ 根据循环统计基本语句次数
- ◆ 用递归关系统计基本语句次数
- ◆ 用平摊方法统计基本语句次数

选取基本语句



统计基本语句频数



计算并简化统计结果

渐近复杂度:

O 、 Ω 、 Θ

标准:

- 平均时间复杂
- 最坏时间复杂
- 最好时间复杂度

基本技术:

- 常用求和公式
- 定积分近似求和
- 递归方程求解

目录

第一节

背景介绍

第二节

算法基本概念

第三节

算法时间复杂度分析

第四节

一个算法例子：
文本距离

一个算法例子：文本相似度分析



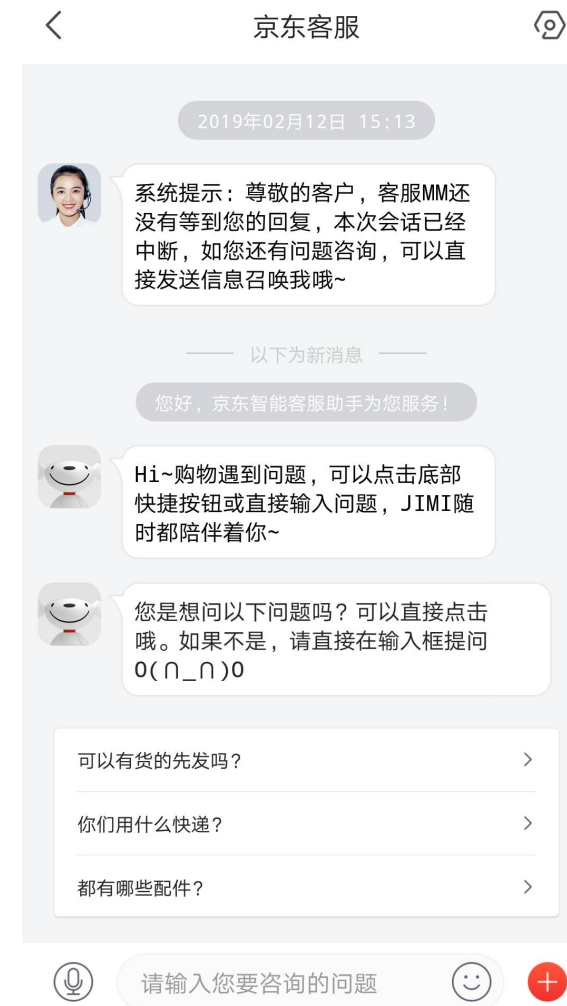
- Given two documents, how similar are they?

- Applications:

- Find similar documents
- Detect plagiarism /duplicates
- Web search

(one “document” is query)

- 自动客服



一个算法例子：文本相似度分析

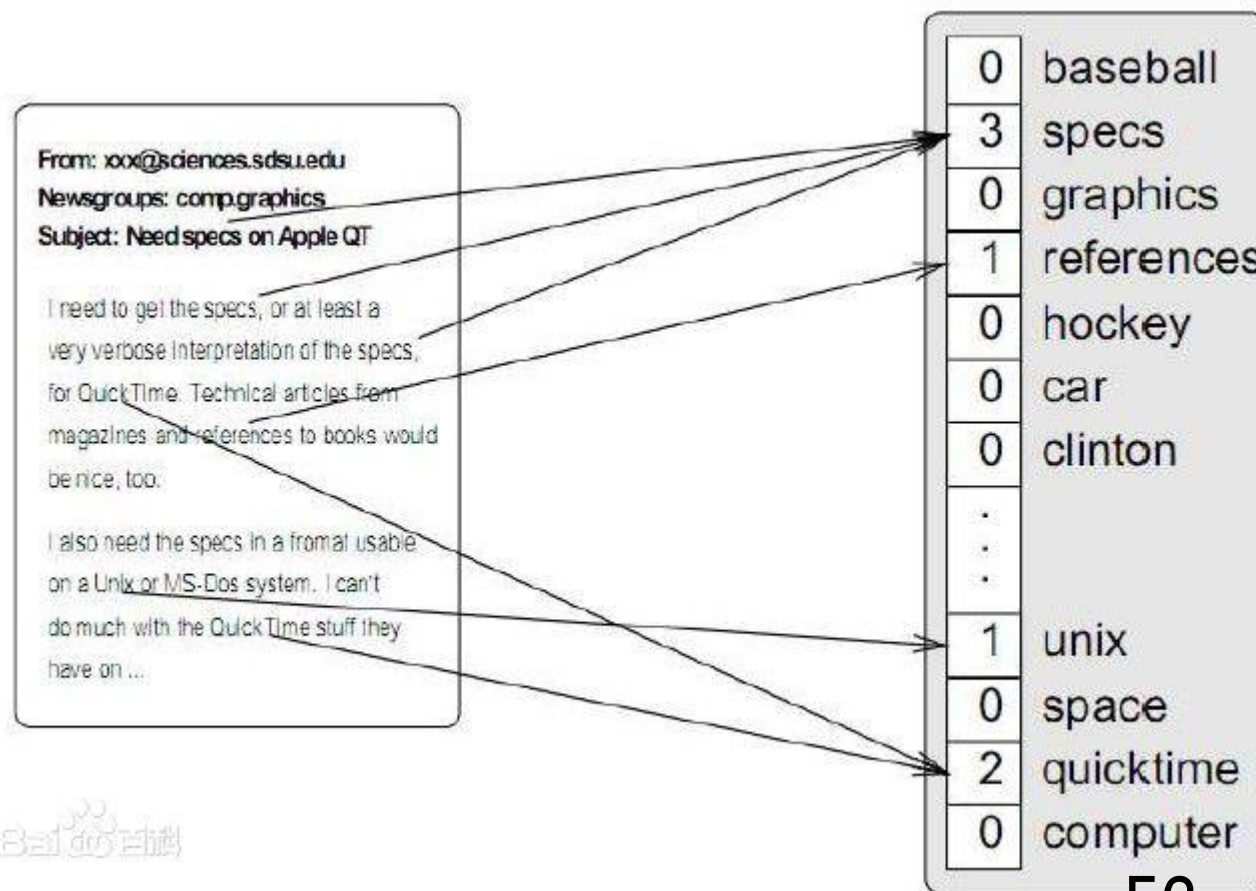


一个算法例子：文本相似度分析



□ 向量空间模型(Vector Space Model, VSM)

- How to define “document”?
- **Word** = sequence of alphanumeric characters
- **Document** = sequence of words
 - Ignore punctuation & formatting



一个算法例子：文本相似度分析



□ 向量空间模型(Vector Space Model, VSM)

- How to define “document”?
- Idea: focus on shared words

Word frequencies: $-D(W) =$
#occurrences of word W
in document D

一个算法例子：文本相似度分析



□ 向量空间模型(Vector Space Model, VSM)

- Treat each document D as a vector of its words
 - One coordinate $D(w)$ for every possible word w

- Example:

- D_1 = “the cat”

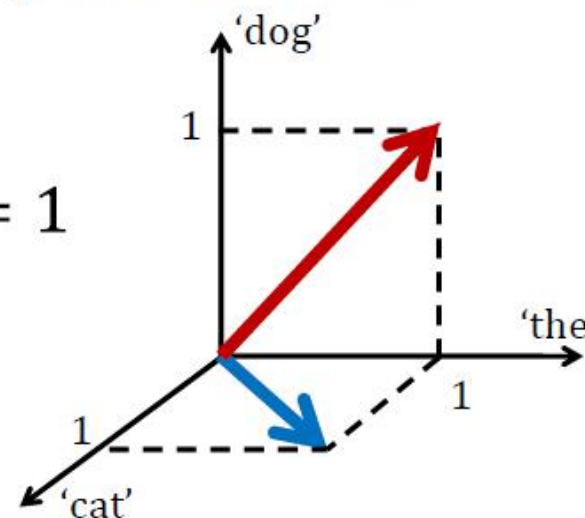
- D_2 = “the dog”

$$D_1 \cdot D_2 = 1$$

- Similarity between vectors?

- **Dot product:**

$$D_1 \cdot D_2 = \sum_w D_1(w) \cdot D_2(w)$$



一个算法例子：文本相似度分析



- Problem: Dot product not scale invariant

- Example 1:

– D_1 = “the cat”

– D_2 = “the dog”

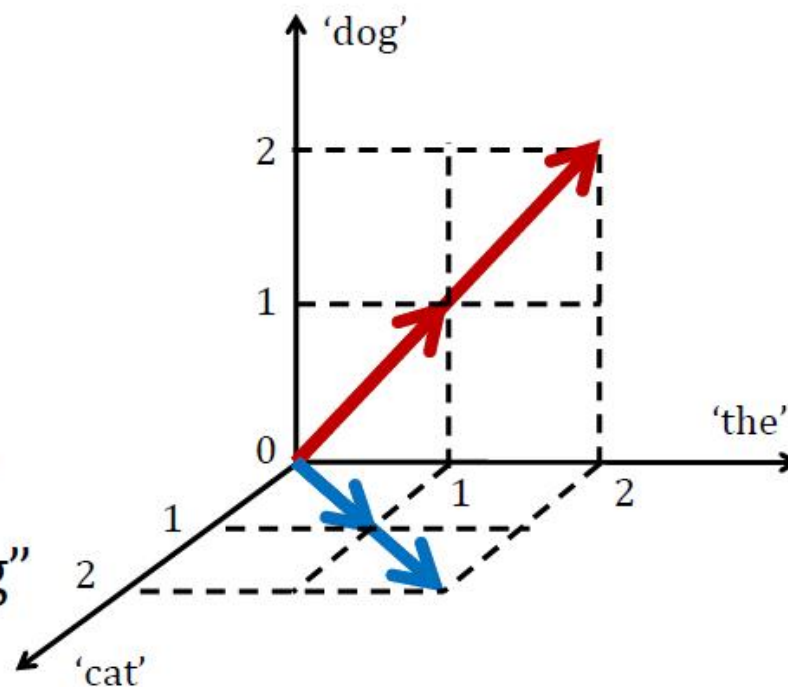
– $D_1 \cdot D_2 = 1$

- Example 2:

– D_1 = “the cat the cat”

– D_2 = “the dog the dog”

– $D_1 \cdot D_2 = 2$



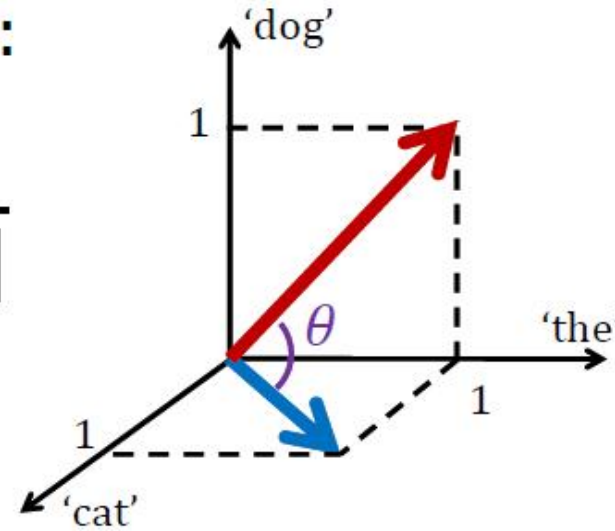
一个算法例子：文本相似度分析



- Idea: Normalize by # words:

$$\cos \theta = \frac{D_1 \cdot D_2}{||D_1|| \cdot ||D_2||}$$

- Geometric solution:
angle between vectors



$$\theta(D_1, D_2) = \arccos \frac{D_1 \cdot D_2}{||D_1|| \cdot ||D_2||}$$

– 0 = “identical”, 90° = orthogonal (no shared words)

一个算法例子：文本相似度分析



□ Algorithm

1. Read documents
2. Split each document into words
3. Count word frequencies (document vectors)
4. Compute document distance

一个算法例子：文本相似度分析



□ Algorithm

1. Read documents

2. Split each document into words

- `re.findall('\w+', doc)`
- But how does this actually work?

3. Count word frequencies (document vectors)

4. Compute document distance

一个算法例子：文本相似度分析



2. Split each document into words

- For each line in document:

- For each character in line:

- If not alphanumeric: Add previous word

- (if any) to list

- Start new word

一个算法例子：文本相似度分析



1. Read documents
2. Split each document into words
- 3. Count word frequencies (document vectors)**
 - a. Sort the word list
 - b. For each word in word list:
 - If same as last word: Increment counter
 - Else: Add last word and its counter to list and reset counter to 0
4. Compute document distance

一个算法例子：文本相似度分析



□ Algorithm

1. Read documents
2. Split each document into words
3. Count word frequencies (document vectors)

4. Compute document distance

For every possible word:

Look up frequency in each document

Multiply

Add to total

一个算法例子：文本相似度分析



1. Read documents
2. Split each document into words
3. Count word frequencies (document vectors)
- 4. Compute document distance**

For every possible word:

Look up frequency in each document

Multiply

Add to total

一个算法例子：文本相似度分析



1. Read documents
2. Split each document into words
3. Count word frequencies (document vectors)
- 4. Compute document distance**
 - a. Start at first word of each document (in sorted order)
 - b. If words are equal:

Multiply word frequencies and Add to total
 - c. In whichever document has lexically lesser word, advance to next word
 - d. Repeat until either document out of words

一个算法例子：文本相似度分析



1. Read documents
2. Split each document into words
3. **Count word frequencies** (document vectors)
 - a. Initialize a dictionary mapping words to counts
 - b. For each word in word list:
 - If in dictionary:
Increment counter
 - Else:
Put 0 in dictionary
4. Compute dot product

一个算法例子：文本相似度分析



1. Read documents
2. Split each document into words
3. Count word frequencies (document vectors)
4. **Compute dot product:**
 - For every word in first document:
 - If it appears in second document:
 - Multiply word frequencies
 - Add to total

算法是什么？总结



- 从哲学角度看：算法是解决一个问题的抽象行为序列。
- 从技术层面上看：算法是一个计算过程，它接受一些输入，并产生某些输出。
- 从抽象层次上看：算法是一个将输入转化为输出的计算步骤序列。
- 从宏观层面上看：算法是解决一个精确定义的计算问题的工具。



湖南大学
HUNAN UNIVERSITY

谢谢观赏

—— 实事求是 敢为人先 ——